DevOps Overview

History of DevOps

- The DevOps movement started to coalesce some time between 2007 and 2008, when IT operations and software development communities raised concerns what they felt was a fatal level of dysfunction in the industry.

- They railed against the traditional software development model, which called for those who write code to be organizationally and functionally apart from those who deploy and support that code.

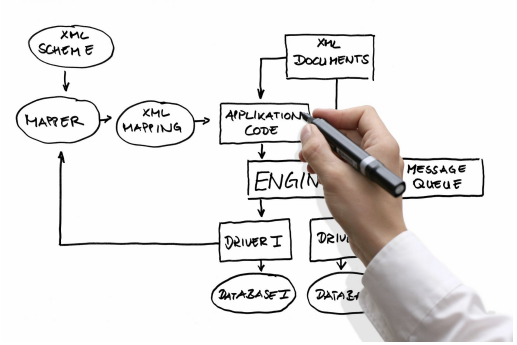
History of DevOps

- Patrick Debois, a Belgian consultant, project manager, and agile practitioner is one among the initiator of DevOps.
- A presentation on “10+ Deploys per Day: Dev and Ops Cooperation at Flickr” helped in bring out the ideas for DevOps and resolve the conflict of “it’s not my code, it’s your machines!”
- DevOps blends lean thinking with agile philosophy.

Why DevOps?

- **Developers and operators do not pursue the same goals.**

Software Development

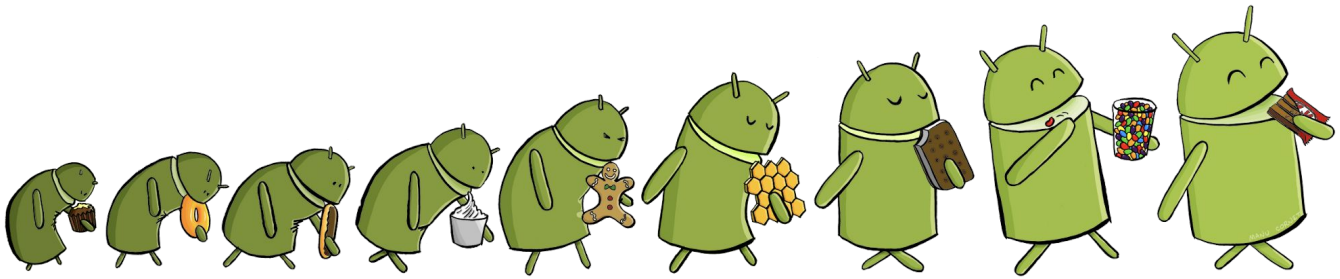


Design and specification

Coding

Testing

Release engineering



Evolution

Software Operation

Monitoring

Troubleshooting

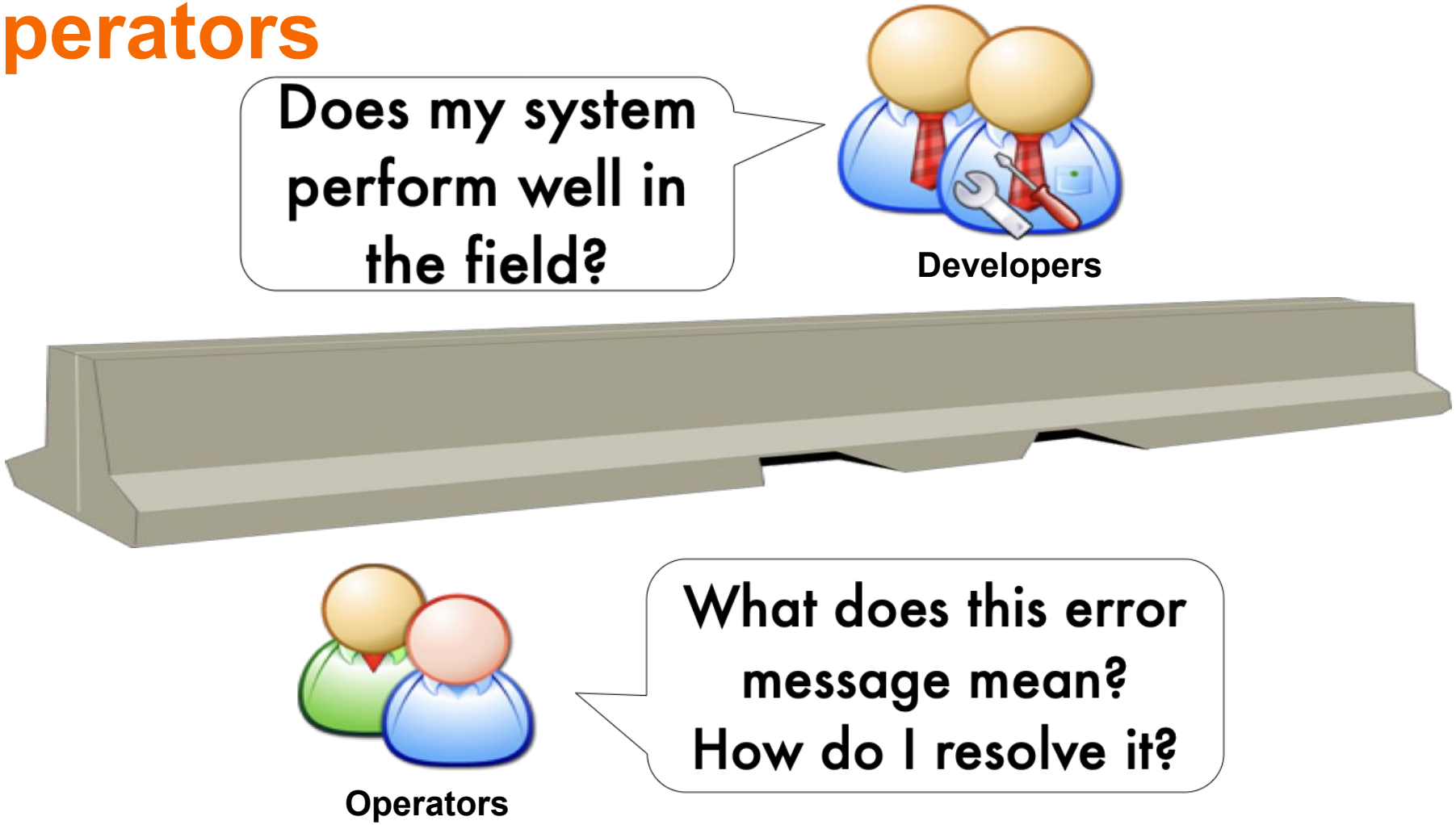
Capacity planning

Anomaly detection

Q&A

Configuration Tuning

There is a gap between software developers and operators



What problem is DevOps trying to solve?

- Poor communication between Dev and Ops
- Opposing goals
  - Devs want to push new features
  - Ops want to keep the system available
    - -> Leads to slow release schedule
- Different cultures
- Limited capacity of operations staff
- Developers have limited insight into operations

Why companies care?

- IBM
  - “IBM has gone from spending about 58% of its development resources on innovation to about 80%”
  - http://devops.com/blogs/ibms-devops-journey/?utm_content=12855120

- Paddy Power (Ireland):
  - “The cycle time from a user story's conception to production has decreased from several months to 2 to 5 days.
  - “Previously, approximately 30% of the workforce was fixing bugs. Now, bugs are so rare that the teams no longer need a bug-tracking system.”

DevOps motivation

- Organizations want to reduce time to market for new features, without sacrificing quality
  - Requires culture change & business-IT alignment
- DevOps practices will influence…
  - the way you organize teams
  - the way you build systems
  - even the structure of the systems that you build
- Unlikely that Devs are able to “throw their final version over the fence” and let operations worry about running it

What is DevOps?

- DevOps is a set of practices intended to reduce the time between committing a change to a system and the change being placed into normal production, while ensuring high quality.

- DevOps is the practice of operations and development engineers participating together in the entire service lifecycle, from design through the development process to production support.

- DevOps is also characterized by operations staff making use many of the same techniques as developers for their systems work.

What is DevOps in practice?

- Continuous collaboration between development & operations.
- Automated CI/CD pipelines, working in small-batches, with shorter lead-times (frequent deployment), and low failure-rates.
- Agile (coding & automation) practices applied to infrastructure, configuration, deployment/releasing and monitoring.

Rule for DevOps Success

Have the “We” culture, rather than “They” culture

Developers

Operators

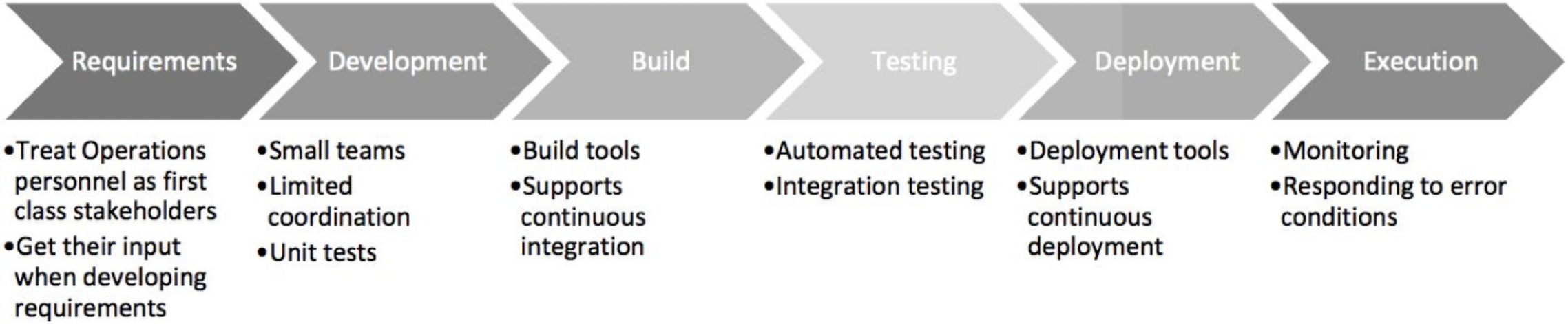
Do not play the blame game, take ownership and share resources

DevOps is to improve communication between developers and operations to solve critical problems like fear of change and risky deployments

What does that mean?

- Quality of the code must be high
  - Testing & test-driven development
- Quality of the build & delivery mechanism must be high
  - Automation & more testing
  - A must when deploying to production 25x per day (etsy.com)
- Time is split:
  - From commit until deployment to production
  - From deployment until acceptance into normal production
  - Means testing in production

DevOps Practices



- Treat Ops as first-class citizens throughout the lifecycle – e.g., in requirements elicitation
  - Many decisions can make operating a system harder or easier
  - Logging and monitoring to suit Ops
- Make Dev more responsible for relevant incident handling
  - Shorten the time between finding and repairing errors

DevOps Practices

- Use continuous deployment, automate everything
  - Commits trigger automatic build, testing, deployment
- Enforce deployment process is used by all
  - No ad-hoc deployments
  - Ensures changes are traceable

DevOps Consequences

Architecturally significant requirement:

Speed up deployment through minimizing synchronous coordination among development teams.

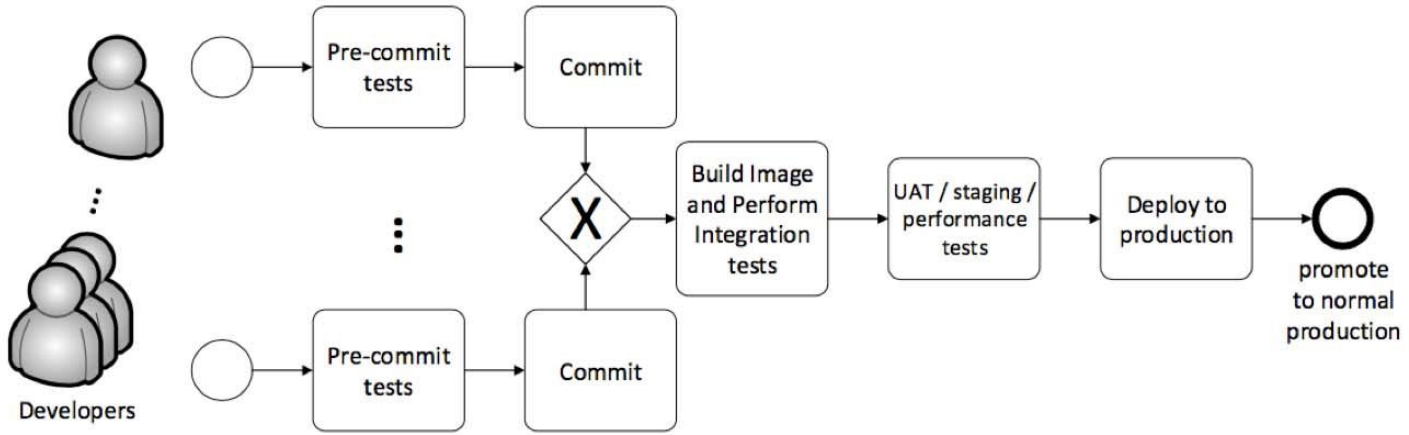
- Synchronous coordination, like a meeting, adds time since it requires
  - Ensuring that all parties are available
  - Ensuring that all parties have the background to make the coordination productive
  - Following up to decisions made during the meeting

DevOps Consequences

- Keep teams relatively small
  - Amazon's "two pizza rule": no team should be larger than can be fed with two pizzas
  - Advantages: make decisions quickly, less coordination overhead, more coherent units

- Team size becomes a major driver of the overall architecture:
  - Small teams develop small services -> Microservices
  - Coordination overhead is minimized by channeling most interaction through service interfaces:
    - Team X provides service A, which is used by teams Y and Z
    - If changes are needed, they are communicated, implemented, and added to the interface.

Continuous Deployment Pipeline



- Developer wants to commit code
- Pre-commit tests are executed locally. If successful:
- Code is committed
- Committed code is compiled, Unit tests are run. If successful:
- Code is built & packaged
- Result can be a machine image or template (assuming virtualization). If successful:
- Integration tests are run. If successful:
- Acceptance / performance tests are run. If successful:
- The new software/service is deployed to production

Thank you!